

Assignment 6: A Search Algorithm for 8-Puzzle

Name: Kaung Khant Lin
Student ID: 6540131

1. Objective

The objective of this assignment is to implement the **A (A-Star) Search algorithm** to solve the 8-puzzle problem. The goal is to find the shortest path from a given random **Start State** to a specific **Goal State** by sliding tiles into the empty space.

The algorithm minimizes the total cost function:

$$F(n) = G(n) + H(n)$$

- **G(n)**: The actual cost (number of moves) from the start state to the current state.
- **H(n)**: The heuristic estimated cost from the current state to the goal state. We use the **Manhattan Distance** for this estimate.

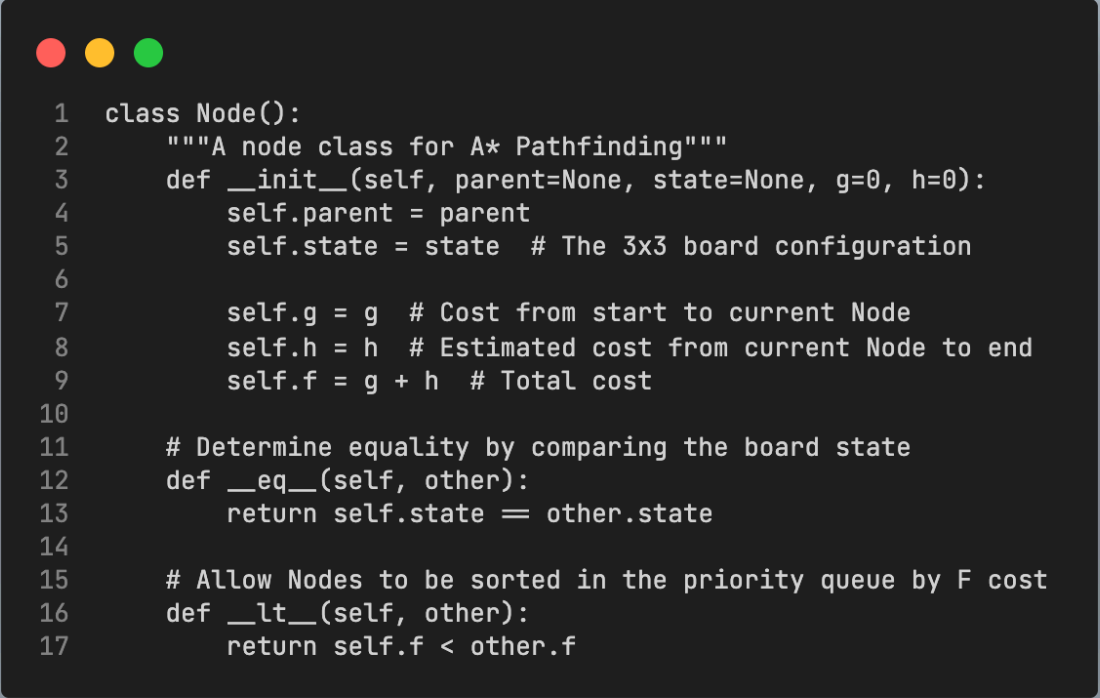
2. Methodology & Implementation

The solution is implemented in Python. The core components of the code are described below.

2.1 The Node Class

The Node class represents a specific state of the puzzle board. It stores the parent node (to trace the solution path), the current board configuration, and the G, H, and F scores.

- Code Snippet:



```
1 class Node():
2     """A node class for A* Pathfinding"""
3     def __init__(self, parent=None, state=None, g=0, h=0):
4         self.parent = parent
5         self.state = state # The 3x3 board configuration
6
7         self.g = g # Cost from start to current Node
8         self.h = h # Estimated cost from current Node to end
9         self.f = g + h # Total cost
10
11     # Determine equality by comparing the board state
12     def __eq__(self, other):
13         return self.state == other.state
14
15     # Allow Nodes to be sorted in the priority queue by F cost
16     def __lt__(self, other):
17         return self.f < other.f
```

2.2 Heuristic Function (Manhattan Distance)

To guide the search efficiently, we calculate the Manhattan Distance. This is the sum of the horizontal and vertical distances of each tile from its target position. It is an admissible heuristic, meaning it never overestimates the cost, guaranteeing the optimal path.

- Code Snippet:

```
1 def manhattan_distance(start_state, goal_state):
2     """Calculate the Manhattan Distance heuristic for the 8-puzzle."""
3     distance = 0
4     # Flatten the tuples for easier indexing if needed, or loop as 2D
5     # Goal positions map: value → (row, col)
6     goal_positions = {}
7     for r, row in enumerate(goal_state):
8         for c, val in enumerate(row):
9             goal_positions[val] = (r, c)
10
11     for r, row in enumerate(start_state):
12         for c, val in enumerate(row):
13             if val != 0: # Don't calculate distance for the empty tile
14                 target_r, target_c = goal_positions[val]
15                 distance += abs(r - target_r) + abs(c - target_c)
16     return distance
```

2.3 A Algorithm Logic

The algorithm uses a **Priority Queue** (heapq) to efficiently select the node with the lowest F cost. It explores valid neighbors by sliding the 'O' tile (blank space) up, down, left, or right.

- Code Snippet:

```

1  def astar(start, end):
2      """Returns a list of board states from start to end"""
3
4      # Create start and end node
5      start_node = Node(None, start, 0, manhattan_distance(start, end))
6
7      # Open list (Priority Queue) and Closed Set
8      open_list = []
9      heapq.heappush(open_list, start_node)
10
11     # Use a set for visited states to speed up lookup
12     closed_set = set()
13
14     # Loop until open list is empty
15     while open_list:
16
17         # Get the current node (heapq automatically pops lowest F)
18         current_node = heapq.heappop(open_list)
19
20         # Found the goal
21         if current_node.state == end:
22             path = []
23             current = current_node
24             total_cost = current.g
25             while current is not None:
26                 path.append(current.state)
27                 current = current.parent
28             return path[::-1], total_cost # Return reversed path
29
30         # Add current state to closed set
31         closed_set.add(current_node.state)
32
33         # Generate children
34         children_states = get_neighbors(current_node.state)
35
36         for child_state in children_states:
37             # If child is already visited, skip
38             if child_state in closed_set:
39                 continue
40
41             # Create the child node
42             g_cost = current_node.g + 1
43             h_cost = manhattan_distance(child_state, end)
44             new_node = Node(current_node, child_state, g_cost, h_cost)
45
46             # Check if this state is already in open_list with a lower G
47             # (Note: simpler implementation just pushes to heap; duplicates handled by closed_set check later)
48             heapq.heappush(open_list, new_node)
49
50     return None, 0

```

3. Test Scenarios and Results

The algorithm was tested on three distinct scenarios to verify its correctness and efficiency.

Scenario 1: Basic Test Case

- **Start State:** ((1, 2, 3), (4, 0, 6), (7, 5, 8))
- **Goal State:** ((1, 2, 3), (4, 5, 6), (7, 8, 0))

Result:

The algorithm successfully found the optimal path.

- **Moves Required:** 2
- **Total Cost:** 2

Output Log:

Move 0:

1 2 3

4 6

7 5 8

Move 1:

1 2 3

4 5 6

7 8

Move 2:

1 2 3

4 5 6

7 8

Scenario 2: Unsolvable Case

- **Start State:** ((8, 7, 2), (4, 0, 5), (3, 1, 6))
- **Goal State:** ((1, 2, 3), (4, 5, 6), (7, 8, 0))

Result:

- **Outcome:** No solution found.

Analysis:

The algorithm correctly identified that no path exists. In 8-puzzle theory, a state is only solvable if the parity of its inversions matches the goal state. The code explored all reachable states and confirmed the goal was unreachable.

Scenario 3: Complex Test Case

- **Start State:** ((8, 2, 4), (7, 1, 6), (3, 0, 5))
- **Goal State:** ((1, 2, 3), (4, 5, 6), (7, 8, 0))

Result:

The algorithm found the solution despite the higher complexity.

- **Moves Required:** 25
- **Total Cost:** 25

Output Log:

Move 0:

```
8 2 4
7 1 6
3  5
```

... [Steps 1-23 omitted for report length] ...

Move 24:

```
1 2 3
4 5
7 8 6
```

Move 25:

```
1 2 3
4 5 6
7 8
```

4. Conclusion

The A* Search implementation was successful.

1. **Correctness:** It correctly solved valid puzzles and identified unsolvable ones.
2. **Efficiency:** By using a priority queue, the algorithm always expanded the most promising nodes first, keeping the search space minimal compared to uninformed search methods like BFS.
3. **Heuristic:** The Manhattan Distance proved effective in guiding the search toward the goal state.

5. Appendix (Full Source Code)

```
import numpy as np
import heapq # Priority queue is faster than sorting a list every time

class Node():
    """A node class for A* Pathfinding"""
    def __init__(self, parent=None, state=None, g=0, h=0):
        self.parent = parent
        self.state = state # The 3x3 board configuration

        self.g = g # Cost from start to current Node
        self.h = h # Estimated cost from current Node to end
        self.f = g + h # Total cost

    # Determine equality by comparing the board state
    def __eq__(self, other):
        return self.state == other.state

    # Allow Nodes to be sorted in the priority queue by F cost
    def __lt__(self, other):
        return self.f < other.f

def manhattan_distance(start_state, goal_state):
    """Calculate the Manhattan Distance heuristic for the 8-puzzle."""
    distance = 0

    # Flatten the tuples for easier indexing if needed, or loop as 2D
    # Goal positions map: value -> (row, col)
    goal_positions = {}
    for r, row in enumerate(goal_state):
        for c, val in enumerate(row):
            goal_positions[val] = (r, c)

    for r, row in enumerate(start_state):
```

```

        for c, val in enumerate(row):
            if val != 0: # Don't calculate distance for the empty tile
                target_r, target_c = goal_positions[val]
                distance += abs(r - target_r) + abs(c - target_c)
    return distance

def get_neighbors(state):
    """Generate all valid neighbor states by sliding the 0 tile."""
    neighbors = []

    # Convert tuple of tuples to list of lists for mutability
    board = [list(row) for row in state]

    # Find the position of 0 (empty tile)
    zero_row, zero_col = -1, -1
    for r in range(3):
        for c in range(3):
            if board[r][c] == 0:
                zero_row, zero_col = r, c
                break

    # Possible moves: Up, Down, Left, Right
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        new_row, new_col = zero_row + dr, zero_col + dc

        # Check bounds
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            # Create a deep copy of the board to swap
            new_board = [row[:] for row in board]

            # Swap 0 with the target neighbor
            new_board[zero_row][zero_col], new_board[new_row][new_col] = \

```



```

        new_board[new_row][new_col], new_board[zero_row][zero_col]

        # Convert back to tuple for hashability/storage
        neighbors.append(tuple(tuple(row) for row in new_board))

    return neighbors

def astar(start, end):
    """Returns a list of board states from start to end"""

    # Create start and end node
    start_node = Node(None, start, 0, manhattan_distance(start, end))

    # Open list (Priority Queue) and Closed Set
    open_list = []
    heapq.heappush(open_list, start_node)

    # Use a set for visited states to speed up lookup
    closed_set = set()

    # Loop until open list is empty
    while open_list:

        # Get the current node (heapq automatically pops lowest F)
        current_node = heapq.heappop(open_list)

        # Found the goal
        if current_node.state == end:
            path = []
            current = current_node
            total_cost = current.g
            while current is not None:
                path.append(current.state)
                current = current.parent

```

```

        return path[::-1], total_cost # Return reversed path

    # Add current state to closed set
    closed_set.add(current_node.state)

    # Generate children
    children_states = get_neighbors(current_node.state)

    for child_state in children_states:
        # If child is already visited, skip
        if child_state in closed_set:
            continue

        # Create the child node
        g_cost = current_node.g + 1
        h_cost = manhattan_distance(child_state, end)
        new_node = Node(current_node, child_state, g_cost, h_cost)

        # Check if this state is already in open_list with a lower G
        # (Note: simpler implementation just pushes to heap; duplicates
handled by closed_set check later)
        heapq.heappush(open_list, new_node)

    return None, 0

def print_board(board):
    for row in board:
        # Create a list of strings: convert numbers to text, replace '0'
with a space
        formatted_row = [str(num) if num != 0 else " " for num in row]
        # Join them with a space to separate columns and print
        print(" ".join(formatted_row))
    print()

```

```
def main():
    """Main function to run A* algorithm on a given 8-puzzle problem."""
    start = (
        (8, 2, 4),
        (7, 1, 6),
        (3, 0, 5)
    )
    end = (
        (1, 2, 3),
        (4, 5, 6),
        (7, 8, 0)
    )

    print("Searching for solution...")
    path, total_cost = astar(start, end)

    if path:
        print(f"Solution found in {len(path) - 1} moves with total cost {total_cost}:\n")
        for i, board in enumerate(path):
            print(f"Move {i}:")
            print_board(board)
    else:
        print("No solution found.")

if __name__ == '__main__':
    main()
```